

4 Controlling the Robot via Processing

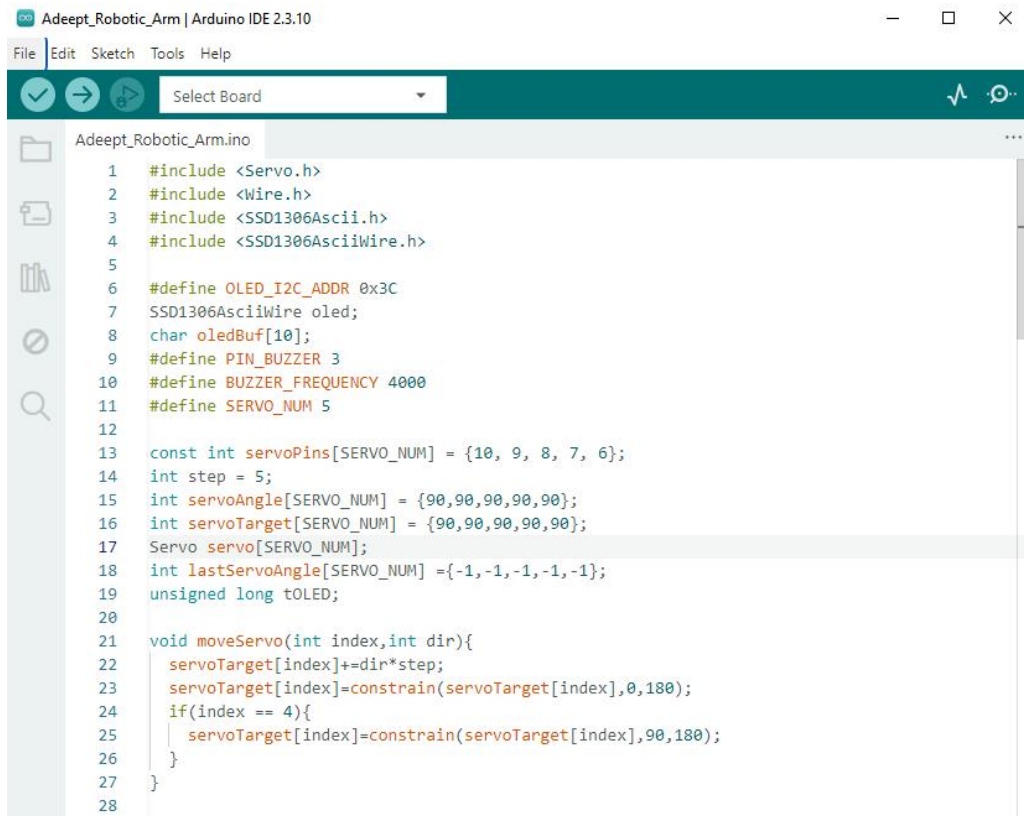
4.1 Overview

In this section, we will learn how to control a robotic arm with Processing. You can operate the robotic arm using your mouse or keyboard. As an extended learning exercise, you will also study inverse kinematics and implement simple writing and drawing motions.

Note: Before getting started, please make sure you understand the principles of ESP8266 wireless communication and have installed the Processing software, as we will be using both of them in the following steps. If you have not covered this content yet, please go through this tutorial first. Tutorial path: **01 Adeept_Robotic_Arm_Control_Board\3 Sensor_Example_Tutorials\Lesson 11 How to Perform Wireless Communication with ESP8266.pdf.**

4.2 Upload infrared control program

1. Connect your computer and Adeept Robotic Arm Board with a USB cable. Then connect the battery and turn on the power switch.
2. Open "04_Control_Processing\Adeept_Robotic_Arm" folder in "/Code", double-click "Adeept_Robotic_Arm".



```

Adept_Robotic_Arm | Arduino IDE 2.3.10
File Edit Sketch Tools Help
Select Board

Adept_Robotic_Arm.ino
1 #include <Servo.h>
2 #include <Wire.h>
3 #include <SSD1306Ascii.h>
4 #include <SSD1306AsciiWire.h>
5
6 #define OLED_I2C_ADDR 0x3C
7 SSD1306AsciiWire oled;
8 char oledBuf[10];
9 #define PIN_BUZZER 3
10 #define BUZZER_FREQUENCY 4000
11 #define SERVO_NUM 5
12
13 const int servoPins[SERVO_NUM] = {10, 9, 8, 7, 6};
14 int step = 5;
15 int servoAngle[SERVO_NUM] = {90,90,90,90,90};
16 int servoTarget[SERVO_NUM] = {90,90,90,90,90};
17 Servo servo[SERVO_NUM];
18 int lastServoAngle[SERVO_NUM] ={-1,-1,-1,-1,-1};
19 unsigned long toLED;
20
21 void moveServo(int index,int dir){
22     servoTarget[index]+=dir*step;
23     servoTarget[index]=constrain(servoTarget[index],0,180);
24     if(index == 4){
25         servoTarget[index]=constrain(servoTarget[index],90,180);
26     }
27 }
28

```

3. Select development board and serial port.

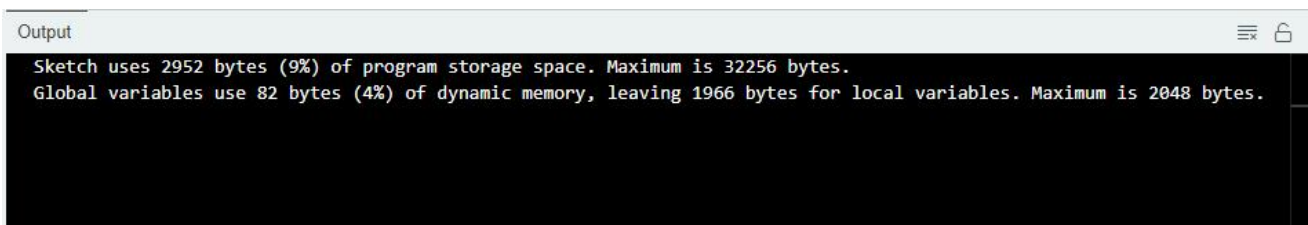
Board: **Tools---**>**Board---**>**Arduino AVR Boards---**>**Arduino Uno**

Port: **Tools --->Port---**>**COMx**

Note: The port number will be different in different computers.



4. After opening, click  to upload the code program to the Arduino. If there is no error warning in the console below, it means that the Upload is successful.

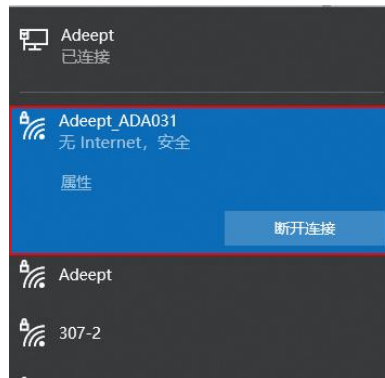


```

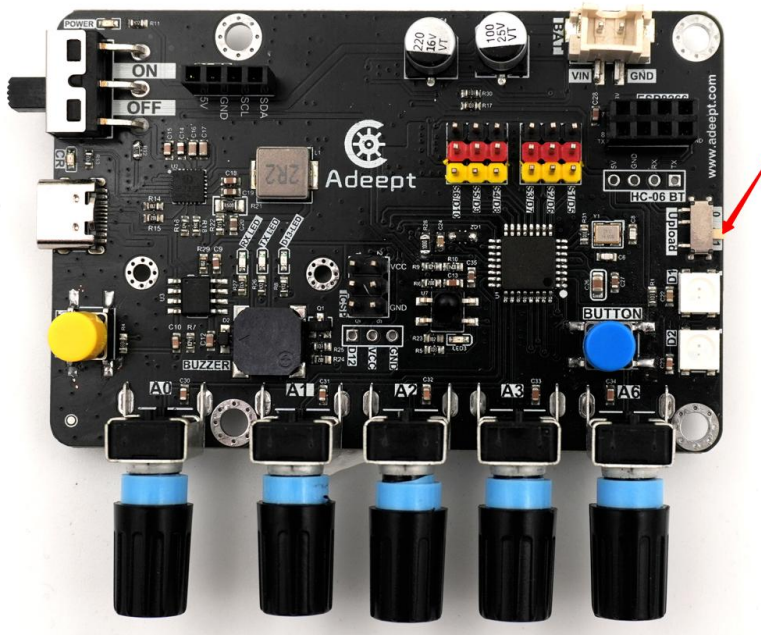
Output
Sketch uses 2952 bytes (9%) of program storage space. Maximum is 32256 bytes.
Global variables use 82 bytes (4%) of dynamic memory, leaving 1966 bytes for local variables. Maximum is 2048 bytes.

```

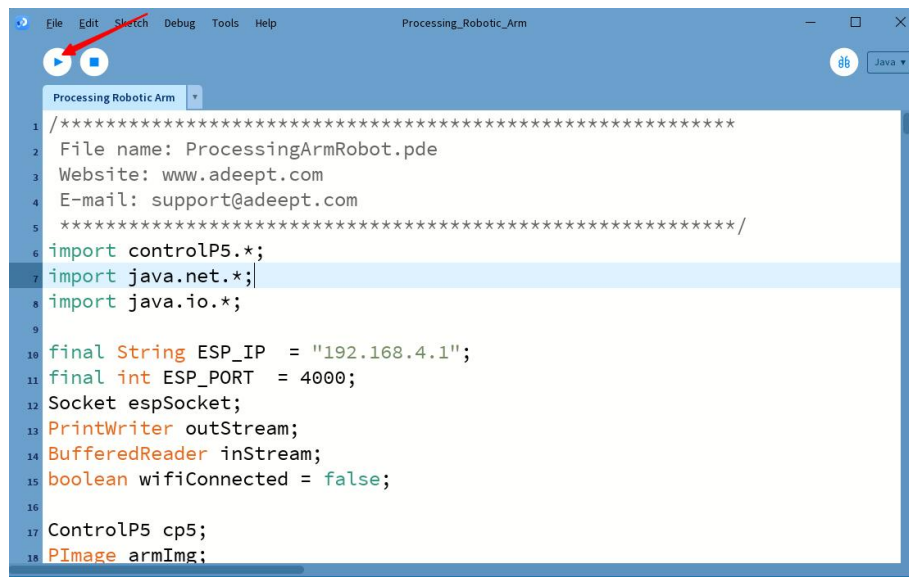
5. After the upload completes successfully, the control board will beep twice to indicate that the program has started. A WiFi hotspot named "**Adept_ADA031**" will appear with the password:
12345678.



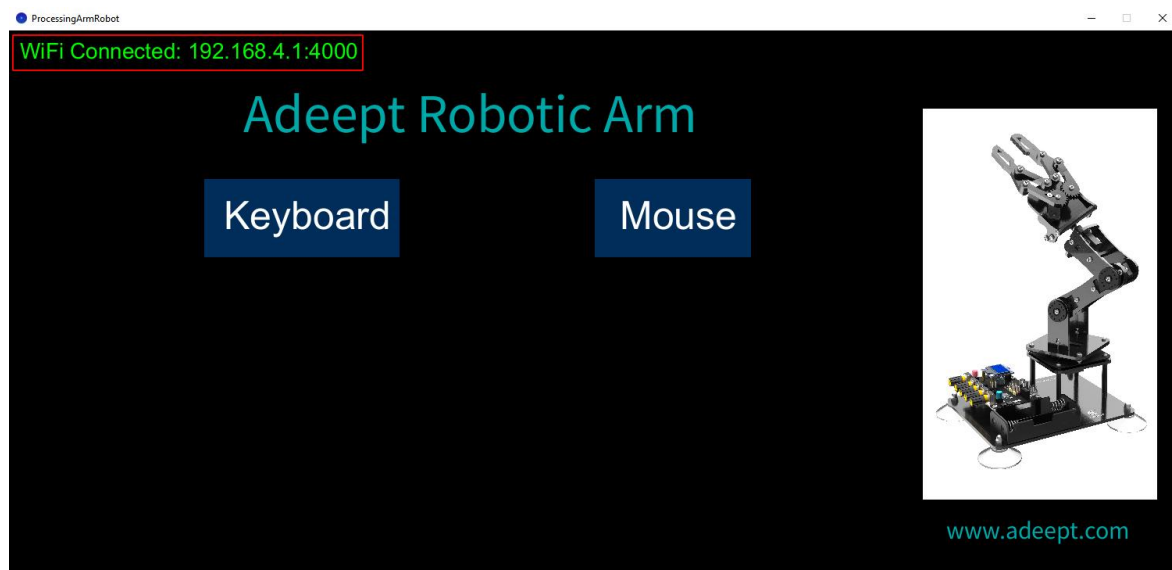
6. Connect your computer to this hotspot, then toggle the upload switch from position **0** to **1** to switch the communication mode to ESP8266.



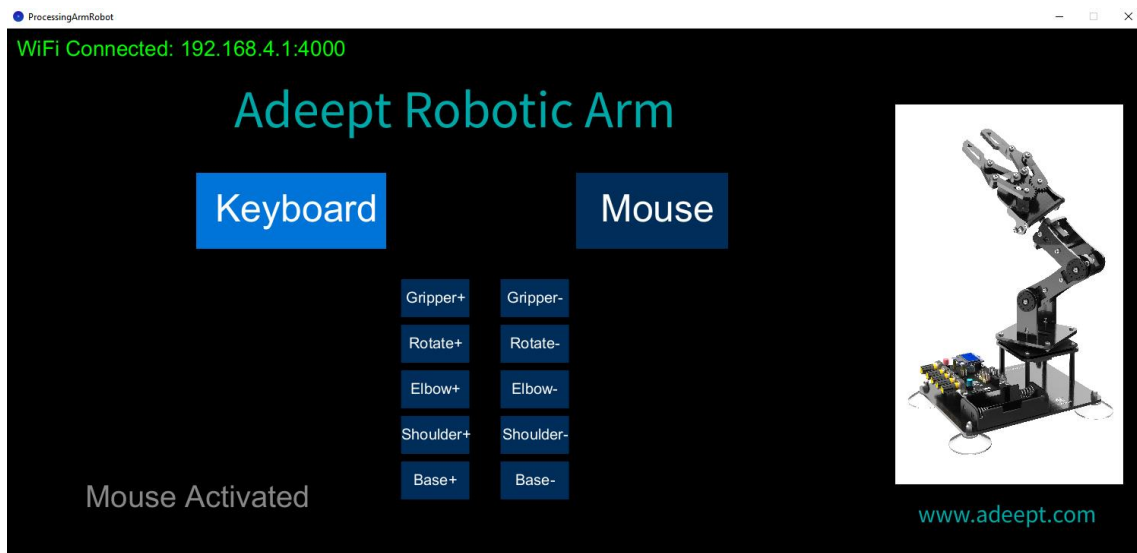
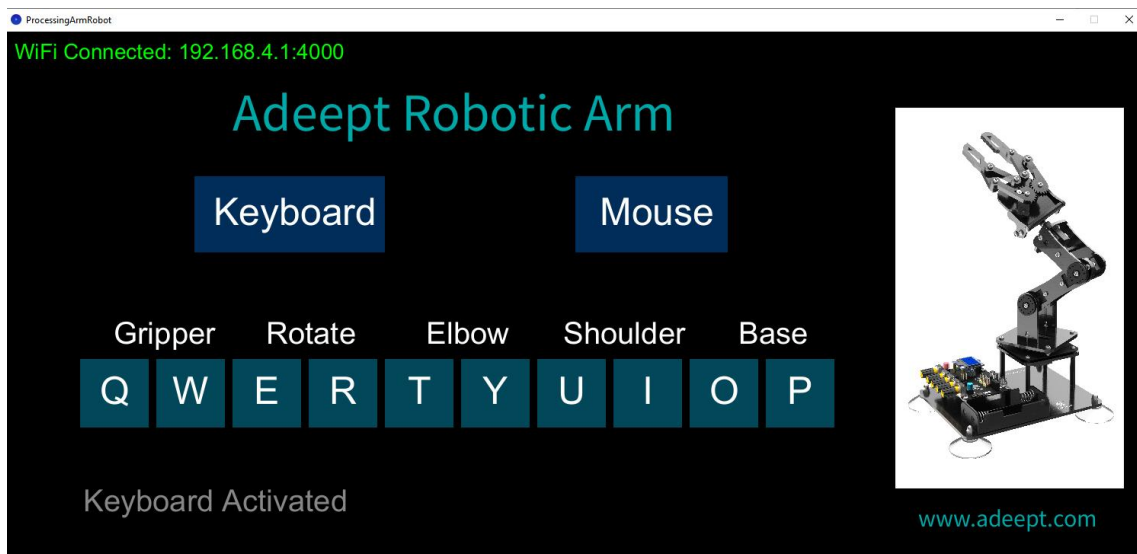
7. Navigate to the directory "**\04_Control_Processing\Processing_Robotic_Arm**", and open the file "**Processing_Robotic_Arm.pde**" with the Processing software.



8. Click Run. Once the interface opens, it will look like the figure shown below. The Wi- Fi connection status is displayed in the top- left corner.



9. The robotic arm supports two control modes: keyboard and mouse. You may choose either according to your preference.



4.3 Code

```

001 #include <Servo.h>
002 #include <Wire.h>
003 #include <SSD1306Ascii.h>
004 #include <SSD1306AsciiWire.h>
005
006 #define OLED_I2C_ADDR 0x3C
007 SSD1306AsciiWire oled;
008 char oledBuf[10];
009 #define PIN_BUZZER 3
010 #define BUZZER_FREQUENCY 4000
011 #define SERVO_NUM 5

```

```

012
013 const int servoPins[SERVO_NUM] = {10, 9, 8, 7, 6};
014 int step = 5;
015 int servoAngle[SERVO_NUM] = {90,90,90,90,90};
016 int servoTarget[SERVO_NUM] = {90,90,90,90,90};
017 Servo servo[SERVO_NUM];
018 int lastServoAngle[SERVO_NUM] ={-1,-1,-1,-1,-1};
019 unsigned long tOLED;
020
021 void moveServo(int index,int dir){
022     servoTarget[index]+=dir*step;
023     servoTarget[index]=constrain(servoTarget[index],0,180);
024     if(index == 4){
025         servoTarget[index]=constrain(servoTarget[index],90,180);
026     }
027 }
028
029 void updateServo(){
030     for(int i=0;i<SERVO_NUM;i++){
031         if(servoAngle[i]<servoTarget[i]) servoAngle[i]++;
032         else if(servoAngle[i]>servoTarget[i]) servoAngle[i]--;
033         servo[i].write(servoAngle[i]);
034     }
035 }
036
037 void setup()
038 {
039     Serial.begin(115200);
040
041     Serial.println("AT+CWMODE=3\r\n");//set to softAP+station mode
042     delay(3000);    //delay 3s
043     Serial.println("AT+CWSAP=\"Adeept_ADA031\", \"12345678\",8,2\r\n");    //TCP Protocol, server IP addr,
044     port
045     delay(1000);    //delay 1s
046     Serial.println("AT+RST\r\n");    //reset wifi
047     delay(1000);    //delay 1s
048     Serial.println("AT+CIPMUX=1\r\n");//set to multi-connection mode
049     delay(1000);
050     Serial.println("AT+CIPSERVER=1,4000\r\n");//set as server
051     delay(1000);
052     Serial.println("AT+CIPSTO=7000\r\n");//keep the wifi connecting 7000 seconds
053     delay(1000);
054
055     for(int i=0;i<SERVO_NUM;i++)
056     {
057         servo[i].attach(servoPins[i]);
058         servo[i].write(servoAngle[i]);
059     }
060
061     oledInit();
062     oledUpdate();
063
064     Buzzer_Setup();    //Buzzer initialization
065     Buzzer_Alert(2, 1);
066 }
067

```

```
068
069 void handleCommand(char cmd)
070 {
071     switch(cmd)
072     {
073         case 'o':
074         case 'O':
075             moveServo(0,+1);
076             break;
077         case 'p':
078         case 'P':
079             moveServo(0,-1);
080             break;
081
082         case 'u':
083         case 'U':
084             moveServo(1,+1);
085             break;
086         case 'i':
087         case 'I':
088             moveServo(1,-1);
089             break;
090
091         case 't':
092         case 'T':
093             moveServo(2,+1);
094             break;
095         case 'y':
096         case 'Y':
097             moveServo(2,-1);
098             break;
099
100         case 'e':
101         case 'E':
102             moveServo(3,+1);
103             break;
104         case 'r':
105         case 'R':
106             moveServo(3,-1);
107             break;
108
109         case 'q':
110         case 'Q':
111             moveServo(4,+1);
112             break;
113         case 'w':
114         case 'W':
115             moveServo(4,-1);
116             break;
117
118     }
119 }
120
121
122 void loop()
123 {
```

```
124 while(Serial.available()){
125     char c = Serial.read();
126     if(c==':'){
127         while(!Serial.available());
128         char cmd=Serial.read();
129         handleCommand(cmd);
130     }
131 }
132 updateServo();
133 oledUpdate();
134 delay(10);
135 }
136
137
138 void Buzzer_Setup(){
139     pinMode(PIN_BUZZER, OUTPUT);
140     digitalWrite(PIN_BUZZER, LOW);
141 }
142
143 void softTone(int durationMs){
144     unsigned long start = millis();
145     int delayUs = 500000 / BUZZER_FREQUENCY; // 半周期
146     while (millis() - start < durationMs) {
147         digitalWrite(PIN_BUZZER, HIGH);
148         delayMicroseconds(delayUs);
149         digitalWrite(PIN_BUZZER, LOW);
150         delayMicroseconds(delayUs);
151     }
152 }
153
154 void Buzzer_Alert(int beat, int rebeat){
155     beat = constrain(beat, 1, 9);
156     rebeat = constrain(rebeat, 1, 255);
157     for (int j = 0; j < rebeat; j++) {
158         for (int i = 0; i < beat; i++) {
159             softTone(100);
160             delay(100);
161         }
162         delay(500);
163     }
164 }
165
166 void oledInit(){
167     Wire.begin();
168
169     oled.begin(&Adafruit128x64, OLED_I2C_ADDR);
170     oled.setFont(Adafruit5x7);
171     oled.clear();
172     oled.setIX();
173
174     oled.setCursor(10,0);
175     oled.print("Adeept Robotic Arm");
176
177     oled.setCursor(10,3);
178     oled.print("S6:");
179     oled.setCursor(75,3);
```



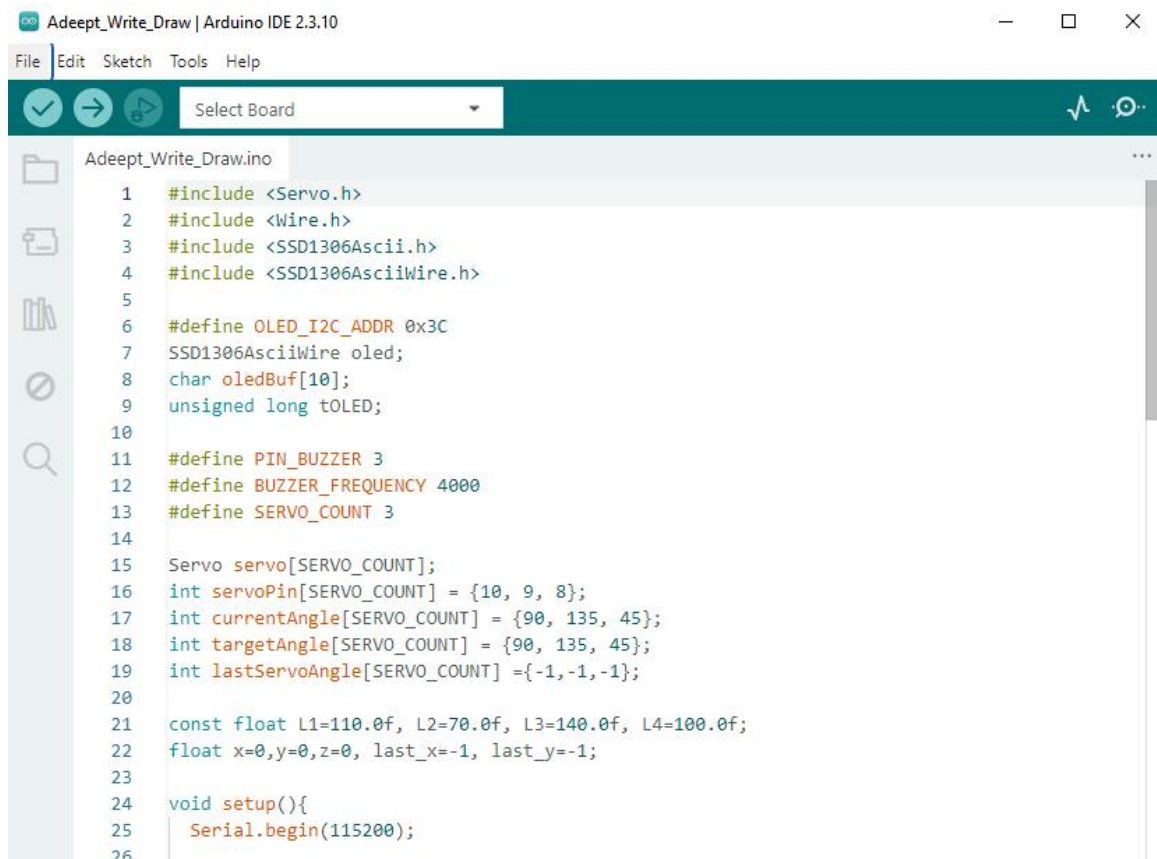
```
180     oled.print("S5:");
181
182     oled.setCursor(10,5);
183     oled.print("S4:");
184     oled.setCursor(75,5);
185     oled.print("S3:");
186
187     oled.setCursor(10,7);
188     oled.print("S2:");
189 }
190
191 void oledPrintNum(int x,int y,int num){
192     oled.set1X();
193     oled.setCursor(x,y);
194     oled.print("  ");
195     oled.setCursor(x,y);
196     oled.print(itoa(num,oledBuf,10));
197 }
198
199 void oledUpdate(){
200     if(millis()-tOLED<100)return;
201     tOLED=millis();
202     if(lastServoAngle[0]!=servoAngle[0])
203     {
204         oledPrintNum(30,3,servoAngle[0]);
205         lastServoAngle[0]=servoAngle[0];
206     }
207
208     if(lastServoAngle[1]!=servoAngle[1])
209     {
210         oledPrintNum(95,3,servoAngle[1]);
211         lastServoAngle[1]=servoAngle[1];
212     }
213
214     if(lastServoAngle[2]!=servoAngle[2])
215     {
216         oledPrintNum(30,5,servoAngle[2]);
217         lastServoAngle[2]=servoAngle[2];
218     }
219
220     if(lastServoAngle[3]!=servoAngle[3])
221     {
222         oledPrintNum(95,5,servoAngle[3]);
223         lastServoAngle[3]=servoAngle[3];
224     }
225
226     if(lastServoAngle[4]!=servoAngle[4])
227     {
228         oledPrintNum(30,7,servoAngle[4]);
229         lastServoAngle[4]=servoAngle[4];
230     }
231 }
```

4.4 Extension - Introduction to Writing and Drawing Mode

1. Before proceeding, please replace the gripper of the robotic arm with the fixing- plate according to the assembly tutorial, so that you can secure the pen in place.



2. Open the directory **"Code\05_Write_and_Draw\Adept_Write_Draw"** and double- click to open the file **"Adept_Write_Draw.ino"** .



```

Adept_Write_Draw.ino
1  #include <Servo.h>
2  #include <Wire.h>
3  #include <SSD1306Ascii.h>
4  #include <SSD1306AsciiWire.h>
5
6  #define OLED_I2C_ADDR 0x3C
7  SSD1306AsciiWire oled;
8  char oledBuf[10];
9  unsigned long toLED;
10
11 #define PIN_BUZZER 3
12 #define BUZZER_FREQUENCY 4000
13 #define SERVO_COUNT 3
14
15 Servo servo[SERVO_COUNT];
16 int servoPin[SERVO_COUNT] = {10, 9, 8};
17 int currentAngle[SERVO_COUNT] = {90, 135, 45};
18 int targetAngle[SERVO_COUNT] = {90, 135, 45};
19 int lastServoAngle[SERVO_COUNT] = {-1, -1, -1};
20
21 const float L1=110.0f, L2=70.0f, L3=140.0f, L4=100.0f;
22 float x=0,y=0,z=0, last_x=-1, last_y=-1;
23
24 void setup(){
25     Serial.begin(115200);
26

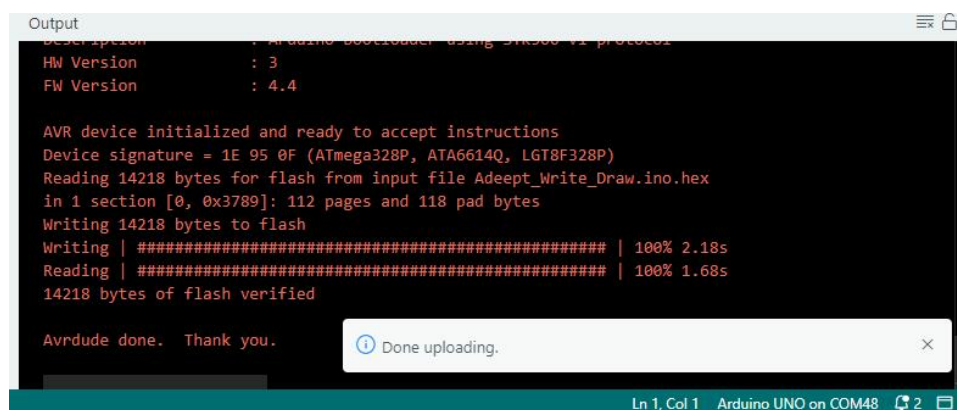
```

3. Select development board and serial port.

4. Board: **Tools---**>**Board---**>**Arduino AVR Boards---**>**Arduino Uno**

5. Port: **Tools --->Port---**>**COMx**

6. Click "**Upload**" to flash the program and wait for the process to complete.



```

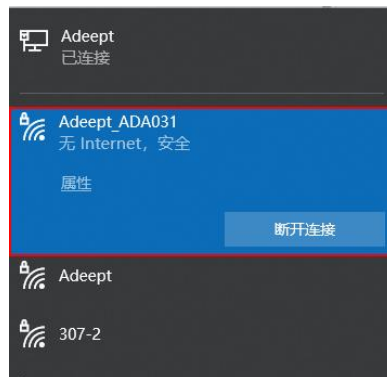
Output
Description      : Arduino Uno (using ATmega328P)
HW Version       : 3
FW Version       : 4.4

AVR device initialized and ready to accept instructions
Device signature = 1E 95 0F (ATmega328P, ATmega164Q, LGT8F328P)
Reading 14218 bytes for flash from input file Adept_Write_Draw.ino.hex
in 1 section [0, 0x3789]: 112 pages and 118 pad bytes
Writing 14218 bytes to flash
Writing | ##### | 100% 2.18s
Reading | ##### | 100% 1.68s
14218 bytes of flash verified

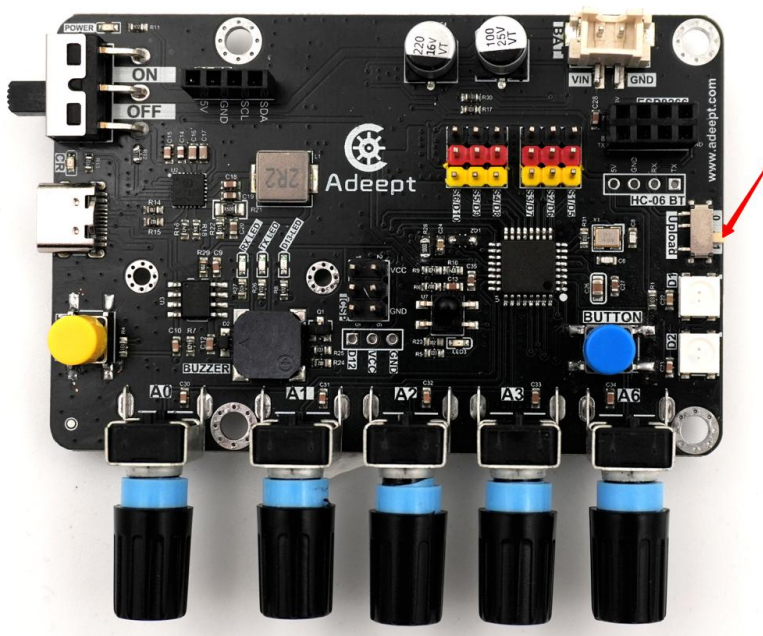
Avrdude done. Thank you.
Done uploading.
Ln 1, Col 1  Arduino UNO on COM48

```

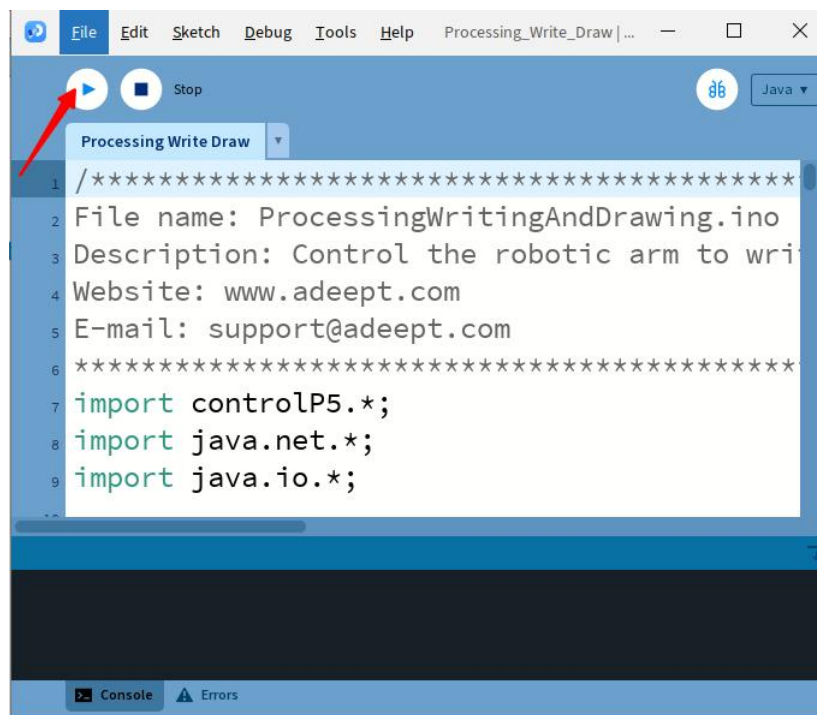
7. After the upload completes successfully, the control board will beep twice to indicate that the program has started. A WiFi hotspot named "**Adeept_ADA031**" will appear with the password: **12345678**.



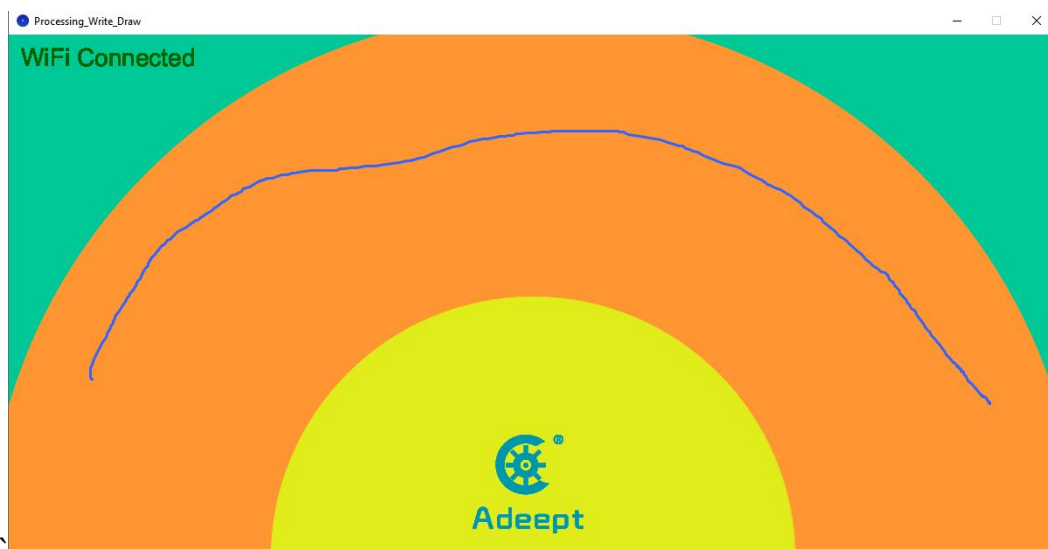
8. Connect your computer to this hotspot, then toggle the upload switch from position **0** to **1** to switch the communication mode to ESP8266.



9. Navigate to the directory "**\05_Write_and_Draw\Adeept_Write_Draw**", and open the file "**Adeept_Write_Draw.pde**" with the Processing software.



10. Click Run. The interface shown below will pop up. The WiFi connection status is displayed in the top- left corner. The orange area represents the workable drawing zone. You can draw with your mouse inside this area, and the robotic arm will move synchronously following your mouse movements.



4.5 Working Principle

This function is implemented based on inverse kinematics. It converts the coordinate system from the Processing software interface to the robotic- arm coordinate system, and then calculates the rotation angle for each servo according to the converted coordinates. The workflow is: Mouse Coordinates → Robotic Arm Coordinates → Servo Angles → Servo Control. Please refer to the code below for details.

Adeapt_Write_Draw.ino

```

001 #include <Servo.h>
002 #include <Wire.h>
003 #include <SSD1306Ascii.h>
004 #include <SSD1306AsciiWire.h>
005
006 #define OLED_I2C_ADDR 0x3C
007 SSD1306AsciiWire oled;
008 char oledBuf[10];
009 unsigned long toLED;
010
011 #define PIN_BUZZER 3
012 #define BUZZER_FREQUENCY 4000
013 #define SERVO_COUNT 3
014
015 Servo servo[SERVO_COUNT];
016 int servoPin[SERVO_COUNT] = {10, 9, 8};
017 int currentAngle[SERVO_COUNT] = {90, 135, 45};
018 int targetAngle[SERVO_COUNT] = {90, 135, 45};
019 int lastServoAngle[SERVO_COUNT] = {-1, -1, -1};
020
021 const float L1=110.0f, L2=70.0f, L3=140.0f, L4=100.0f;
022 float x=0,y=0,z=0, last_x=-1, last_y=-1;
023
024 void setup(){
025     Serial.begin(115200);
026
027     Serial.println("AT+CWMODE=3\r\n");//set to softAP+station mode
028     delay(3000); //delay 3s
029     Serial.println("AT+CWSAP=\"Adeapt_ADA031\", \"12345678\", 8, 2\r\n"); //TCP Protocol, server IP addr,
030     port
031     delay(1000); //delay 1s
032     Serial.println("AT+RST\r\n"); //reset wifi
033     delay(1000); //delay 1s
034     Serial.println("AT+CIPMUX=1\r\n");//set to multi-connection mode
035     delay(1000);
036     Serial.println("AT+CIPSERVER=1,4000\r\n");//set as server

```

```
037   delay(1000);
038   Serial.println("AT+CIPSTO=7000\r\n");//keep the wifi connecting 7000 seconds
039   delay(1000);
040
041   for(byte i=0;i<SERVO_COUNT;i++){
042     servo[i].attach(servoPin[i]);
043     servo[i].write(currentAngle[i]);
044   }
045
046   oledInit();
047   oledUpdate();
048
049   Buzzer_Setup();
050   Buzzer_Alert(2, 1);
051 }
052
053
054 void loop(){
055   if (Serial.available()) {
056     String str = Serial.readStringUntil('\n');
057     if (parseXYZ(str, x, y, z)) {
058       inverseKinematics(x,y,z,targetAngle[0],targetAngle[1],targetAngle[2]);
059     }
060   }
061   updateServo();
062   oledUpdate();
063   delay(10);
064 }
065
066 bool parseXYZ(String str, float &x, float &y, float &z){
067   str.trim();
068   int pos = str.indexOf(':');
069   if (pos != -1) str = str.substring(pos + 1);
070   int i1 = str.indexOf(',');
071   int i2 = str.indexOf(',', i1 + 1);
072   if (i1 == -1 || i2 == -1) return false;
073
074   x = str.substring(0, i1).toFloat();
075   y = str.substring(i1 + 1, i2).toFloat();
076   z = str.substring(i2 + 1).toFloat();
077   return true;
078 }
079
080 void updateServo(){
081   for(int i=0;i<SERVO_COUNT;i++){
082     if(currentAngle[i]<targetAngle[i]) currentAngle[i]++;
083     else if(currentAngle[i]>targetAngle[i]) currentAngle[i]--;
084     servo[i].write(currentAngle[i]);
085   }
086 }
087
088 void Buzzer_Setup(){
089   pinMode(PIN_BUZZER, OUTPUT);
090   digitalWrite(PIN_BUZZER, LOW);
091 }
092
```

```
093 void softTone(int durationMs){
094     unsigned long start = millis();
095     int delayUs = 500000 / BUZZER_FREQUENCY;
096     while (millis() - start < durationMs) {
097         digitalWrite(PIN_BUZZER, HIGH);
098         delayMicroseconds(delayUs);
099         digitalWrite(PIN_BUZZER, LOW);
100         delayMicroseconds(delayUs);
101     }
102 }
103
104 void Buzzer_Alert(int beat, int rebeat){
105     beat = constrain(beat, 1, 9);
106     rebeat = constrain(rebeat, 1, 255);
107     for (int j = 0; j < rebeat; j++) {
108         for (int i = 0; i < beat; i++) {
109             softTone(100);
110             delay(100);
111         }
112         delay(500);
113     }
114 }
115
116 void oledInit(){
117     Wire.begin();
118
119     oled.begin(&Adafruit128x64, OLED_I2C_ADDR);
120     oled.setFont(Adafruit5x7);
121     oled.clear();
122     oled.set1X();
123
124     oled.setCursor(10,0);
125     oled.print("Adeept Robotic Arm");
126
127     oled.setCursor(10,3);
128     oled.print("S6:");
129     oled.setCursor(10,5);
130     oled.print("S5:");
131     oled.setCursor(10,7);
132     oled.print("S4:");
133
134     oled.setCursor(75,3);
135     oled.print("x:");
136     oled.setCursor(75,5);
137     oled.print("y:");
138 }
139
140 void oledPrintNum(int x,int y,int num){
141     oled.set1X();
142     oled.setCursor(x,y);
143     oled.print("    ");
144     oled.setCursor(x,y);
145     oled.print(itoa(num,oledBuf,10));
146 }
147
148 void oledUpdate(){
```



```

149   if(millis()-tOLED<100) return;
150   tOLED=millis();
151
152   if(lastServoAngle[0]!=currentAngle[0]){
153       oledPrintNum(30,3,currentAngle[0]);
154       lastServoAngle[0]=currentAngle[0];
155   }
156   if(lastServoAngle[1]!=currentAngle[1]){
157       oledPrintNum(30,5,currentAngle[1]);
158       lastServoAngle[1]=currentAngle[1];
159   }
160   if(lastServoAngle[2]!=currentAngle[2]){
161       oledPrintNum(30,7,currentAngle[2]);
162       lastServoAngle[2]=currentAngle[2];
163   }
164   if(last_x != x || last_y !=y){
165       oledPrintNum(90,3,x);
166       oledPrintNum(90,5,y);
167       last_x = x;
168       last_y = y;
169   }
170 }
171
172 bool inverseKinematics(float x,float y,float z,int &servo1,int &servo2,int &servo3){
173     float theta1 = atan2(y,x);
174     float px = sqrt(x*x+y*y);
175     float pz = z - L1;
176     float D = sqrt(px*px+pz*pz);
177     if(D > L2+L3 || D < fabs(L2-L3)) return false;
178
179     float beta = atan2(pz,px);
180     float cos_gamma = (L2*L2 + D*D - L3*L3) / (2*L2*D);
181     cos_gamma = constrain(cos_gamma,-1.0f,1.0f);
182     float gamma = acos(cos_gamma);
183     // elbow up
184     float theta2 = PI - gamma - beta;
185     float cos_phi = (D*D+L3*L3-L2*L2) / (2*D*L3);
186     cos_phi = constrain(cos_phi,-1.0f,1.0f);
187     float theta3 = theta2 - beta + acos(cos_phi);
188     if((90-degrees(theta1)<0 || 90-degrees(theta1)>180) || (degrees(theta2) + 15<0 ||
189 degrees(theta2)+15>180) || (degrees(theta3)-90<0 || degrees(theta3)-90>180)) return false;
190     servo1 = 90-degrees(theta1);
191     servo2 = degrees(theta2) + 15;
192     servo3 = degrees(theta3)-90;
193     return true;
194 }

```

Processing_Write_Draw.pde

```
01  /*****
02  File name: ProcessingWritingAndDrawing.ino
03  Description: Control the robotic arm to write and draw through the upper computer interface written by
04  Processing.
05  Website: www.adeept.com
06  E-mail: support@adeept.com
07  *****/
08  import controlP5.*;
09  import java.net.*;
10  import java.io.*;
11
12  final String ESP_IP = "192.168.4.1";
13  final int ESP_PORT = 4000;
14  Socket espSocket;
15  PrintWriter outStream;
16  BufferedReader inStream;
17  boolean wifiConnected = false;
18  Textlabel wifiStatusLabel;
19  ControlP5 cp5;
20  PImage img;
21
22  void connectESP8266() {
23      closeWiFiSocket();
24      try {
25          espSocket = new Socket(ESP_IP, ESP_PORT);
26          outStream = new PrintWriter(espSocket.getOutputStream(), true);
27          inStream = new BufferedReader(new InputStreamReader(espSocket.getInputStream()));
28          wifiConnected = true;
29          println("WiFi Connected -> " + ESP_IP + ":" + ESP_PORT);
30      } catch (Exception e) {
31          wifiConnected = false;
32          println("ESP Connect Failed, check IP/ESP WiFi");
33      }
34  }
35
36  void closeWiFiSocket(){
37      try{
38          if(inStream != null) inStream.close();
39          if(outStream != null) outStream.close();
40          if(espSocket != null) espSocket.close();
41      }catch(Exception e){
42          println("Stream close failed");
43      }
44  }
45
46  void setup() {
47      size(1200, 600);
48      background(0,200,150);
49      fill(255, 150, 50);
50      stroke(255, 150, 50);
51      ellipse(600, 600, 1250, 1250);
52      fill(224,237,26);
53      stroke(224,237,26);
```

```
54 ellipse(600, 600, 600,600);
55 img = loadImage("LOGO1.png");
56
57 cp5 = new ControlP5(this);
58 connectESP8266();
59 wifiStatusLabel = cp5.addTextlabel("wifiStatus").setPosition(10, 15).setFont(createFont("Arial",
60 28));
61 }
62
63 void draw() {
64   stroke(50,100,255);
65   strokeWeight(3);
66   image(img, 530, 450);
67
68   if(wifiConnected){
69     wifiStatusLabel.setText("WiFi Connected").setColorValue(0xff006400);
70   }else{
71     wifiStatusLabel.setText("WiFi Disconnected").setColorValue(0xffff3333);
72   }
73
74   if (mousePressed) {
75     boolean now = dist(mouseX, mouseY, 600, 600) <= 625 && dist(mouseX, mouseY, 600, 600) >= 300;
76     boolean pre = dist(pmouseX, pmouseY, 600, 600) <= 625 && dist(pmouseX, pmouseY, 600, 600) >= 300;
77     if (now && pre) {
78       line(mouseX, mouseY, pmouseX, pmouseY);
79       print("mouseX:"+ mouseX + "   mouseY:"+ mouseY);
80       float px = height - mouseY;
81       float py = mouseX - width/2.0;
82       float x = map(px, 0, height, 0, 210);
83       float y = map(py, -width/2.0, width/2.0, -210, 210);
84       float z= 100;
85       println("  x:"+ x + "    y:"+ y);
86       sendXYZ(x,y,z);
87     }
88   }
89 }
90
91 void sendXYZ(float x,float y,float z){
92   if(wifiConnected){
93     outStream.print(String.format("%.2f,%.2f,%.2f\n", x,y,z));
94     outStream.flush();
95   }
96 }
```